A FROTHY V0.1 FIELD GUIDE

Frothy From the Ground Up

A two-day tour through the language, the runtime, and the image that makes it feel alive.

FROTHY PROJECT · EDITION 2026.04

Contents

Learn Frothy in 30 Minutes

- Literals and values
- Top-level names are stable slots
- Simple rebinding
- Calling code
- Locals and block structure
- set and mutation
- if, when, and unless
- repeat and while
- fn and to
- cells(n) creation, indexing, and mutation
- words, see, core, and slotInfo
- save(), restore(), and wipe()
- One small board and FFI example
- Redefining a top-level slot changes what old callers observe

How to Read This Guide in Two Days

Part I: Using Frothy

- Frothy's Core Mental Model and What Changed from Froth
- The Runtime Value Model: What a Value Is in Frothy
- Stable Top-Level Slot Identity, Base Image vs Overlay Image, and Rebinding Semantics
- Running Frothy in Practice
- Frothy Projects in Practice
- Build and Flash Workflows

Part II: How Frothy Works

- Memory, Ownership, Lifetimes, Refcounts, Capacities, Cellspace Usage, and Reset Stories
- Surface Syntax, Parser Structure, Lexical Resolution, and Why Code Is Non-Capturing
- Canonical IR: Node Types, Literals, Links, Rendering, and Why IR Is the Persisted Code Truth
- Evaluator Architecture: Frames, Locals, Slot Lookup, Calls, Mutation, Loops, Interrupts, and Safe Points
- Cells: Why They Are Narrow, What They Can Store, How Indexing Works, and Why They Are Top-Level Owned
- Persistence: save, restore, wipe, Snapshot Layout, Symbol/Object Tables, Pointer-Free Restore, and Failure Handling
- Inspection: words, see, core, slotInfo, and How Inspection Reflects Canonical IR Rather Than Raw Source
- Interactive Profile: Boot, Safe Boot, REPL, Multiline Input, Prompt Behavior, Ctrl-C, Recovery
- Control Transport: .control Mode, Single-Owner Sessions, Framed Requests/Events, Helper/Editor Story

Part III: Extending Frothy

- FFI Boundary: Native Bindings, Board Slots, Allowed Types, Non-Persistable State, Substrate Reuse
- Creating Project FFI in Practice
- Creating Board FFI in the Repo
- Tests, Proofs, and Validation Surface: What Is Actually Proved
- Current Follow-On Queue: What Has Landed, What Is Queued Next, and What Is Still Only Draft Work

Learn Frothy in 30 Minutes

This is the fast tour. The rest of the guide turns you into a maintainer, but this opening section is meant to get you reading and writing Frothy right now.

Scope note:

- The language contract in this guide follows accepted Frothy `v0.1` semantics.
- Most interactive examples use the current spoken-ledger surface that the shipped parser and shell accept today, because that is what you will actually type at the prompt.
- Frothy is not teaching a user-visible data stack first. If you know old Froth or Forth, treat that as background substrate knowledge, not as the product model you should teach from.

One quick orientation before the examples:

- A **value** is a thing like `7`, `true`, `"tea"`, some `Code`, or some `Cells`.
- A **slot** is a stable top-level named place. The slot named `count` stays `count`; the value inside it can change.
- A **block** is code inside `[` and `]`.
- The REPL prints the value an expression returns. A mutation usually returns `nil`.

Literals and values

```
42
true
false
"tea"
nil
```

RESULT

- `42`
- `true`
- `false`
- `"tea"`
- `nil`

Frothy starts from values you can see directly. There is no stack juggling to understand before you can read a line of code.

Top-level names are stable slots

```
count is 7
count
```

> **RESULT**
>
> - `count is 7` creates or rebinds the top-level slot named `count`
> - `count` evaluates to `7`

Think of `count` as a named box in the live image. You are not creating a temporary variable that disappears at the end of the line.

Simple rebinding

```
count is 7
count is 8
count
```

> **RESULT**
>
> - the slot named `count` still exists
> - the value inside it is now `8`

That distinction matters all through Frothy: slot identity is stable, but the value inside the slot can be replaced.

Calling code

The `to` form defines top-level code directly:

```
to inc with x [ x + 1 ]
inc: 41
```

> **RESULT**
>
> - `inc` is now bound to a `Code` value
> - `inc: 41` returns `42`

You can also build `Code` as a value with `fn`:

```
double is fn with x [ x * 2 ]
double: 21
```

> **RESULT**
> - `double` holds a `Code` value
> - `double: 21` returns `42`

If the thing you are calling is itself produced by code, use `call`:

```
makeInc is fn [ fn with x [ x + 1 ] ]
call makeInc: with 41
```

> **RESULT**
> - `makeInc:` returns a `Code` value
> - `call ... with ...` invokes that returned code
> - the whole expression returns `42`

Locals and block structure

Blocks use `[` and `]`.

Inside a block, `name is value` creates a local when that name is not already a top-level slot in play, and `here` makes the intent especially obvious:

```
localDemo is fn [
  here n is 5
  n
]

localDemo:
```

> **RESULT**
> - `localDemo:` returns `5`
> - `n` exists only while `localDemo` is running

Blocks can hold several expressions. The last one is the block's result.

```
mathDemo is fn [
  here a is 20
  here b is 22
  a + b
]

mathDemo:
```

> **RESULT**
> - 42

`set` and mutation

Use `set` when the place already exists and you want to change it.

```
count is 7
set count to 8
count
```

> **RESULT**
> - `set count to 8` returns `nil`
> - `count` now returns 8

`set` also works on locals:

```
bump is fn [
  here n is 10
  set n to n + 1
  n
]

bump:
```

> **RESULT**
> - 11

`if`, `when`, and `unless`

Use `if` when you want both branches:

```
choose is fn with flag [
  if flag [ "yes" ] else [ "no" ]
]

choose: true
choose: false
```

> **RESULT**
> - "yes"
> - "no"

Use `when` when there is only a then-branch:

```
when true [ 42 ]
when false [ 42 ]
```

> **RESULT**
>
> - 42
> - nil

Use `unless` for the opposite condition:

```
unless false [ 42 ]
unless true [ 42 ]
```

> **RESULT**
>
> - 42
> - nil

`repeat` and `while`

`repeat` is the friendly counted loop:

```
counter is cells(1)
set counter[0] to 0
repeat 3 as i [ set counter[0] to counter[0] + i ]
counter[0]
```

> **RESULT**
>
> - after the loop, `counter[0]` is `3`
> - the loop index `i` takes the values `0`, `1`, and `2`

`while` is the lower-level loop:

```
loopDemo is fn [
  here i is 0
  while i < 3 [
    set i to i + 1
  ]
  i
]

loopDemo:
```

> **RESULT**
> - 3

`fn` and `to`

Use `to` when you are defining a named top-level function:

```
to square with x [ x * x ]
square: 9
```

Use `fn` when code is just another value:

```
squareToo is fn with x [ x * x ]
squareToo: 9
```

> **RESULT**
> - both return 81

The difference is not “one is real code and the other is not”. Both produce `Code`. The difference is where the binding happens.

`cells(n)` creation, indexing, and mutation

`Cells` are Frothy’s small, narrow array value in `v0.1`.

```
frame is cells(2)
set frame[0] to "left"
set frame[1] to 99
frame[0]
frame[1]
```

> **RESULT**
> - `"left"`
> - 99

In `v0.1`, `Cells` can store:

- `Int`
- `Bool`
- `Nil`
- `Text`

They do not store other `Cells`, `Code`, or native bindings.

`words`, `see`, `core`, and `slotInfo`

First define something worth inspecting:

```
demo is fn [ when true [ 42 ] ]
```

List the names that currently exist:

```
words()
```

Typical result includes names such as:

- `save`
- `restore`
- `wipe`
- `words`
- `see`
- `core`
- `slotInfo`
- board names like `LED_BUILTIN` and `gpio.write`
- your own names like `demo`

Show the normalized surface form of a binding:

```
see("demo")
```

Result:

```
demo | overlay | code
to demo [ when true [ 42 ] ]
```

Show the canonical core form that the evaluator actually runs:

```
core("demo")
```

Result:

```
(fn arity=0 locals=0 (seq (if (lit true) (seq (lit 42)))))
```

Ask what kind of thing a slot is:

```
slotInfo("demo")
slotInfo("save")
```

Result:

```
demo | overlay | code | persistable | user
save | base | native | non-persistable | foreign
```

That one pair of lines already teaches a lot:

- `demo` lives in the overlay image, stores `Code`, and can be persisted
- `save` is part of the base image, is a native binding, and is not persistable

`save()`, `restore()`, and `wipe()`

These are the image controls.

```
note is "draft"
save()
set note to "edited"
restore()
note
wipe()
note
```

> **RESULT**
>
> - `save()` returns `nil`
> - `restore()` returns `nil`
> - after `restore()`, `note` is back to `"draft"`
> - `wipe()` returns `nil`
> - after `wipe()`, `note` is gone and reading it raises `eval error (4)`

What they mean:

- `save()` snapshots the current overlay image
- `restore()` replaces the current overlay with the saved one
- `wipe()` clears both the live overlay and the saved snapshot

One small board and FFI example

FFI means *foreign function interface*: the narrow bridge between Frothy values and native C or board code.

On the POSIX board, GPIO calls print traces so you can see what would happen on hardware. On a real ESP32 target, the same names hit the board driver.

```
gpio.mode: LED_BUILTIN, 1
gpio.write: LED_BUILTIN, 1
ms: 50
gpio.write: LED_BUILTIN, 0
```

Typical POSIX result:

```
[gpio] pin 2 -> OUTPUT
nil
[gpio] pin 2 = HIGH
nil
nil
[gpio] pin 2 = LOW
nil
```

And a read-style call:

```
adc.read: A0
```

Result on POSIX:

- a deterministic integer such as `2048`

Redefining a top-level slot changes what old callers observe

This is one of the central Frothy ideas.

```
message is "first"
to sayMessage [ message ]
sayMessage:
message is "second"
sayMessage:
```

> **RESULT**
>
> - first call returns `"first"`
> - second call returns `"second"`

Why? Because `sayMessage` does not capture a frozen copy of `message`. It reads the stable top-level slot each time it runs.

Now that you can read and write basic Frothy, here is how Frothy is actually built.

How to Read This Guide in Two Days

If you only have two or three days, do not read this like a reference manual. Read it like flight training.

Day 1:

1. Read the quickstart again, but type every example.
2. Read Part I through `Build and flash workflows`.
3. Run Lab 1 and Lab 2.
4. Re-read the chapters on stable slots, persistence, and inspection. Those are the three ideas that unlock the rest of the system.

Day 2:

1. Read Part II in order. The order matters: parser, then IR, then evaluator, then persistence, then control transport.
2. Keep one terminal open with the REPL and another with the codebase.
3. Read every `Code Walk` with the file open beside you.
4. Run Lab 3 if you have hardware.
5. Finish with Part III so the FFI and proof surface stop feeling magical.

If Thursday is close and you need the highest-value path, focus on these sentences until they feel obvious:

- Top-level slot identity is stable; values change, slot identity does not.
- `Code` is non-capturing in `v0.1`.
- Canonical IR, not raw source text, is the persisted truth for code.
- Persistence is overlay-only, not full execution-state persistence.
- The device image is the state. Host tooling is intentionally thin.

Part I: Using Frothy

Frothy's Core Mental Model and What Changed from Froth

Frothy is easier to understand once you stop asking "where did that stack item go?" and start asking "which place does this expression read or write?"

The core pieces are:

- **Values**: integers, booleans, `nil`, text, `Cells`, `Code`, and native bindings.
- **Places**: slots, locals, and indexed `Cells` positions.
- **Stable top-level slots**: named places that survive rebinding.
- **Locals**: short-lived places inside a running block or call.
- **Code**: non-capturing executable values.
- **Overlay image**: the mutable live layer on top of the base image.
- **Base image**: the built-in world the runtime seeds at boot, including `save`, `restore`, `wipe`, inspection words, and filtered board bindings.

Old Froth let the visible stack model dominate how users thought about the system. Frothy deliberately moves the center of gravity. The substrate still exists underneath, especially in the board and FFI layers, but the language you maintain is lexical, named, and image-oriented.

Here is the picture that pays rent:

```
+-----------------------------+
|         Base Image          |
| save restore wipe words ... |
| filtered board slots        |
+--------------+--------------+
               |
               v
+-----------------------------+
|        Overlay Image        |
| your slots and rebound code |
| saved/restored/wiped        |
+--------------+--------------+
               |
               v
+-----------------------------+
|         Slot Table          |
| name -> stable slot id      |
| slot id -> current value    |
+-----------------------------+
```

The slot table is the stable naming layer. The overlay changes the current binding for a slot, but it does not invent a new slot identity each time you rebind.

Code Walk

Start with `src/frothy_base_image.c`.

`frothy_base_image_install()` seeds a fixed list of base slots: snapshot control, inspection words, and the special `boot` slot. It then calls `frothy_ffi_install_board_base_slots()` so the board surface becomes part of the same base world.

`frothy_base_image_reset()` is just as important. It does three jobs, in this order:

1. Release overlay-owned runtime state.
2. Clear any base slot bindings back to the built-in baseline.
3. Reset the heap pointer to the boot watermark and reinstall the base image.

That shape tells you something deep about Frothy's design: reset is not "try to undo the last few edits." Reset is "rebuild the base image and make the overlay earn its way back in."

Worked Example

```
message is "first"
to sayMessage [ message ]
sayMessage:
message is "second"
sayMessage:
```

Execution story:

1. `message is "first"` binds the stable slot `message` to a text value.
2. `to sayMessage [ message ]` stores `Code` in the stable slot `sayMessage`.
3. The first call reads slot `message` and returns `"first"`.
4. `message is "second"` writes a new value into the same slot.
5. The second call reads the same slot again and now returns `"second"`.

What to remember

- Frothy is an image language before it is a file language.
- Slots are stable top-level places.
- The overlay is the live, mutable layer.
- The base image is rebuilt, not patched in place, on reset and wipe.
- Old Froth substrate behavior matters only where Frothy intentionally reuses it.

Common confusions

- "Is `count is 8` creating a new variable?" No. It is writing a new value into the stable top-level slot `count`.
- "Does `Code` capture locals like a closure?" No, not in `v0.1`.
- "Is `wipe()` just clearing RAM?" Not exactly. It clears the overlay image and the saved snapshot, then rebuilds the base image.

Invariants

- A top-level slot's identity is stable once created.
- Base slots must come back after reset, restore failure, or wipe.
- Overlay bindings are the only bindings that persistence saves.
- A slot can change value class over time; the slot identity itself does not.

The Runtime Value Model: What a Value Is in Frothy

At runtime, every Frothy value fits in a 32-bit word. A **tagged value** is a word that carries both payload bits and enough tag information to tell you what kind of thing it is.

Some values are immediate:

- `Int`

- `Bool`
- `Nil`

Some values are references into the runtime's object table:

- `Text`
- `Cells`
- `Code`
- `Native`

The runtime structure in `src/frothy_value.h` makes the model concrete:

- an object table for live heap-like objects
- a payload arena for text and packed code bytes
- a `cellspace` region for `Cells`
- a scratch arena for temporary evaluation values

Diagram:

```
value word
  |
  +-- immediate tag -> int / bool / nil
  |
  +-- object-ref tag -> object table index
                          |
                          +-- TEXT   -> payload span in payload arena
                          +-- CELLS  -> span in cellspace
                          +-- CODE   -> packed IR payload + metadata
                          +-- NATIVE -> C function pointer + context
```

Why this shape?

- It keeps common values cheap.
- It keeps persisted objects pointer-free.
- It makes inspection and snapshotting tractable on small targets.

Code Walk

Read `src/frothy_value.h` and `src/frothy_value.c` together.

`frothy_value_t` is just a `uint32_t`, but the runtime struct around it is the real story. It preallocates bounded storage:

- `FROTHY_OBJECT_CAPACITY` objects

- `FROTHY_PAYLOAD_CAPACITY` bytes of payload
- `FROTHY_EVAL_VALUE_CAPACITY` temporary evaluation slots

This is not a general-purpose heap. It is a deliberately bounded runtime with a clear reset story.

`frothy_value_make_int()`, `frothy_value_make_bool()`, and `frothy_value_make_nil()` create immediates. `frothy_runtime_alloc_text()`, `frothy_runtime_alloc_cells()`, `frothy_runtime_alloc_code()`, and `frothy_runtime_alloc_native()` append live objects. `frothy_value_retain()` and `frothy_value_release()` then maintain object lifetimes with a **refcount** (reference count, meaning "how many owners still point at this object").

Worked Example

```
"frothy"
fn [ 1 ]
cells(1)
```

What happens conceptually:

- `"frothy"` creates a `Text` object backed by the payload arena.
- `fn [ 1 ]` creates a `Code` object backed by packed IR payload.
- `cells(1)` creates a `Cells` object backed by `cellspace`.

Those three values look equally ordinary at the language surface, but the runtime stores them in three different ways because their ownership and persistence stories are different.

What to remember

- Every value is a 32-bit runtime word.
- Some values are immediate; others are object references.
- `Text` and packed `Code` consume payload arena bytes.
- `Cells` consume `cellspace`, not the payload arena.
- Native bindings are values too, but they are not persistable.

Common confusions

- "Is `Code` stored as source text?" No. It is stored as canonical IR and associated metadata.
- "Are `Cells` just text or byte arrays?" No. They are their own object kind with narrow element rules.
- "Does `nil` allocate?" No.

Invariants

- Runtime values are always valid 32-bit tagged words.
- An object reference must point at a live object table entry.
- Releasing the last reference must clear the object and reclaim its storage.
- Persisted representations cannot depend on raw process pointers.

Stable Top-Level Slot Identity, Base Image vs Overlay Image, and Rebinding Semantics

The single most important semantic commitment in Frothy is that top-level names are stable slots, not throwaway dictionary entries.

That means:

- `count is 7` and `count is 8` talk about the same slot
- old callers observe new slot values
- snapshots save the overlay bindings for those slots
- `restore()` rebuilds those bindings onto the same stable names

The base image versus overlay image split is how Frothy makes this practical.

- The **base image** is what boot always knows how to seed.
- The **overlay image** is what the user has changed since boot.

Code Walk

`src/frothy_eval.c` contains `frothy_slot_write_owned()`, the evaluator helper that turns a `WRITE_SLOT` IR node into a real slot mutation.

The function first looks up the slot by name. If the write is a creation-style write, it may create the slot. Then it releases any old bound value, marks the slot as overlay-owned, installs the new value, and updates the slot arity if the new value is `Code` or `Native`.

That last detail is easy to miss, but it matters: rebinding a slot is not only changing a value. It is also updating the calling surface the shell and control tools expect.

Worked Example

```
message is "first"
to sayMessage [ message ]
sayMessage:
message is "second"
sayMessage:
```

This example is worth repeating because it is the simplest proof that Frothy is late-bound at the slot level. The code stored in `sayMessage` does not freeze a copy of `message`. It performs a top-level slot read every time.

What to remember

- Slots are stable; values are replaceable.
- `is` is the create-or-rebind form.
- `set` is the mutate-an-existing-place form.
- Overlay writes are what snapshots save.

Common confusions

- "Why does old code see the new value?" Because top-level slot reads are late reads, not closure captures.
- "Does `restore()` resurrect deleted local state?" No. It rebuilds top-level overlay state only.
- "Can a base slot become overlay?" Yes, by rebinding its value. The slot is still the same slot; the current binding becomes overlay-owned.

Invariants

- A slot name resolves to one stable slot table entry.
- Overlay writes must retain the installed value and release the old one.
- Rebinding must keep slot metadata, especially arity, truthful.
- Base image reset must be able to recover even after partial overlay failure.

Running Frothy in Practice

This chapter is the first hands-on loop. The goal is not to memorize commands. The goal is to make the image model feel real under your fingers.

Hands-on walkthrough

Lab 1: the local Frothy loop

Build the host runtime:

```
make build
```

Run it:

```
make run
```

You can also run the binary directly:

```
build/Frothy
```

A typical startup banner looks like this:

```
Frothy shell
snapshot: none
boot: CTRL-C for safe boot
Type help for commands.
frothy>
```

If you previously saved a snapshot, the second line may say `snapshot: found`.

Now do a clean local session:

```
wipe()
count is 7
to inc with x [ x + 1 ]
inc: 41
demo is fn [ when true [ 42 ] ]
see("demo")
core("demo")
save()
set count to 99
restore()
count
```

What you should notice:

- `wipe()` gives you a known base-only image
- `see("demo")` prints normalized surface code, not whatever source formatting you originally typed
- `core("demo")` prints the evaluator's core tree
- `restore()` brings back the saved overlay

Shell conveniences:

- `words` is a shell command that maps to `words()`

- `show @demo` maps to `see("demo")`
- `core @demo` maps to `core("demo")`
- `info @demo` maps to `slotInfo("demo")`
- `remember` maps to `save()`

The prompt changes to `..` when the shell thinks your input is incomplete.

What can go wrong

- If the banner says `snapshot: found`, you are not starting from a blank image. Run `wipe()` first when teaching or debugging.
- `parse error (...)` means the parser could not build IR from the source.
- `eval error (...)` means parsing succeeded but execution failed, often because a name was missing or a type was wrong.
- `Ctrl-C` interrupts running evaluation and returns control at the next safe point.

What to remember

- The host runtime is not a toy. It is the main teaching and debugging target.
- `wipe()` is the best way to get back to a known state.
- Inspection is part of the normal workflow, not a last resort.

Frothy Projects in Practice

Frothy is an image language, but the CLI gives you a project scaffold so you can version source, dependencies, build settings, and board target choice cleanly.

Inside this repo, the repo-local CLI binary is `tools/cli/froth-cli`. In a packaged release, the user-facing command is `froth`.

Hands-on walkthrough

Lab 2: your first Frothy project

Create a project:

```
tools/cli/froth-cli new hello
```

That writes:

- `froth.toml`

- `src/main.froth`
- `lib/.gitkeep`
- `.gitignore`

The generated manifest looks like this in the important parts:

```
[project]
name = "hello"
version = "0.1.0"
entry = "src/main.froth"

[target]
board = "posix"
platform = "posix"

# [dependencies]
# utils = { path = "lib/utils.froth" }

# [ffi]
# sources = ["src/ffi/bindings.c"]
# includes = ["src/ffi"]
# defines = { MY_CONSTANT = "42" }
```

The project source starts with a tiny boot example:

```
note = nil

boot {
  set note = "Hello from Frothy!"
}
```

That file still uses the older surface style in the scaffold, but the runtime it generates is the same Frothy image model you have been reading about.

Typical project loop:

```
cd hello
tools/cli/froth-cli send
tools/cli/froth-cli build
./.froth-build/firmware/Frothy
```

`froth send` does three important things for project source:

1. resolves includes
2. strips boundary markers
3. wipes the live overlay before loading the project and then runs `boot()` or `autorun()` if present

Dependencies are source-level includes. A named dependency entry in `froth.toml` lets `\ #use "name"` resolve cleanly, but direct relative includes also work.

Useful project commands:

- `froth send`
- `froth build`
- `froth connect`
- `froth connect --local`
- `froth doctor`

What can go wrong

- `froth send` talks to a real device or runtime. If no device is connected, use `froth connect --local` or the host binary directly.
- `froth doctor` is the first thing to run when board setup feels uncertain.
- Include resolution errors usually mean the manifest dependency path or a relative `#use` path is wrong.

What to remember

- `froth.toml` is the project authority.
- Project source is resolved and normalized before it is sent or built.
- The image remains the real state; project files are how you reconstruct it.

Build and Flash Workflows

The CLI supports three practical flows:

- POSIX project build
- ESP-IDF project build and flash
- prebuilt firmware flashing for the default ESP32 board

Hands-on walkthrough

What lands in `.froth-build/` for a POSIX project:

- `resolved.froth`: merged source before boundary stripping
- `runtime.frothy`: source that should be applied to the runtime image
- `firmware/Frothy`: the host executable

- `project_ffi.cmake`: generated only when `[ffi]` is present

For a normal POSIX project:

```
froth build
froth flash
```

`froth flash` on POSIX is intentionally simple. It builds and then prints the binary path because there is no hardware flash step:

```
binary: /path/to/project/.froth-build/firmware/Frothy
```

For an ESP32 project:

```
froth new --target esp32-devkit-v1 blink
cd blink
froth doctor
froth build
froth flash
```

The important implementation detail is this: the CLI no longer tries to bake the whole runtime source into ESP-IDF firmware at build time. Instead it:

1. stages and builds the board firmware
2. flashes the firmware
3. reconnects
4. applies `.froth-build/runtime.frothy` into the live image

That is a better fit for Frothy's image model. Firmware and overlay state are related, but not the same thing.

Outside a project checkout, `froth flash` can also fetch and flash a prebuilt release firmware for the default ESP32 target.

What can go wrong

- If `esp-idf` is missing, `froth doctor` will tell you before `froth build` fails later.
- If no USB-serial port is visible, `froth flash` cannot choose a device.
- If several ports are visible, pass `--port`.
- On ESP32, build success does not mean your overlay source has been applied yet; that happens after flashing.

What to remember

- POSIX flash is a no-op that points at a binary.

- ESP32 flash is firmware first, runtime image second.
- `.froth-build/runtime.frothy` is the bridge between project source and live image state.

Part II: How Frothy Works

Memory, Ownership, Lifetimes, Refcounts, Capacities, Cellspace Usage, and Reset Stories

Frothy's memory model is small on purpose. You are maintaining a bounded live system, not a language runtime that assumes a desktop heap will quietly grow forever.

Important bounded capacities from `src/frothy_value.h`:

- `FROTHY_EVAL_VALUE_CAPACITY 256`
- `FROTHY_OBJECT_CAPACITY 128`
- `FROTHY_PAYLOAD_CAPACITY 16384`

Those are compile-time limits. They are part of the design, not temporary scaffolding.

Ownership works like this:

- slots own the values bound into them
- locals own the values stored in the current frame
- temporary evaluator buffers own child results until they are released
- object values keep live storage alive with refcounts

A **lifetime** is simply how long a piece of state stays valid. Frothy tries to make those lifetimes boring and visible.

Code Walk

`src/frothy_value.c` contains the runtime's cleanup heart: `frothy_runtime_clear_overlay_state()`.

It walks overlay slots and releases their bound values. It resets `cellspace` to base. It discards every live object, clears free-span bookkeeping, resets the payload arena accounting, resets the evaluator scratch arena, and increments `reset_epoch`.

That last field matters in `src/frothy_eval.c`. Evaluation frames and temporary buffers remember the `reset_epoch` they were allocated under. If a reset happens during evaluation, cleanup code can tell it is looking at stale storage and avoid double-release mistakes.

This is exactly what “clear reset story” looks like in systems code. The code is not clever. It is defensive.

Worked Example

Run this in the REPL:

```
wipe()
texty is "hello"
codey is fn [ 1 ]
frame is cells(1)
save()
wipe()
```

What gets released:

- the text payload for `texty`
- the packed code payload for `codey`
- the `Cells` span for `frame`
- any overlay slot bindings that owned those values

What survives:

- the base image
- the ability to rebuild new overlay state from scratch

What to remember

- Bounded capacities are part of the product shape.
- Refcounts are how object lifetimes stay explicit.
- `cellspace` is a separate arena used by `Cells`.
- Reset is a first-class operation, not an error case hack.

Common confusions

- “Does Frothy have a garbage collector?” No. It uses reference counting plus explicit reset paths.
- “Is payload storage the same thing as cellspace?” No. Text and packed code live in the payload arena; `Cells` live in `cellspace`.
- “Why does evaluation have its own scratch arena?” To make temporary allocations bounded and easy to reset.

Invariants

- Every retained object must eventually be released or cleared by reset.
- `cellspace` must return to base on overlay reset.
- After overlay clear, object count, payload usage, and eval scratch usage must all be zeroed.
- `reset_epoch` must advance whenever overlay state is cleared.

Surface Syntax, Parser Structure, Lexical Resolution, and Why `Code` Is Non-Capturing

Frothy's current surface tries to read like spoken code:

- `count is 7`
- `set count to 8`
- `to inc with x [ x + 1 ]`
- `repeat 3 as i [ ... ]`

But the parser is not trying to preserve the exact source the user typed. Its job is to resolve names, lower sugar, and produce canonical IR.

Lexical resolution means "decide at parse time whether a name refers to a local or to a top-level slot." Frothy does that with scopes and frames in `src/frothy_parser.c`.

The crucial design choice is that `Code` is non-capturing in `v0.1`. In plain language: an inner function does not close over outer locals.

Why choose that?

- It keeps persistence straightforward.
- It keeps the evaluator simple.
- It preserves the "top-level slots are stable, locals are temporary" split.
- It avoids inventing a closure object model before the language needs one.

Code Walk

Read two regions in `src/frothy_parser.c`.

`frothy_push_frame()` stores whether a frame rejects outer capture.

`frothy_resolve_name()` first searches locals in the current frame. If it sees a matching name in an outer frame and `reject_outer_capture` is true, it returns a signature error instead of silently creating a closure-like reference.

Then read `frothy_parse_fn_node()`. Every `fn` body pushes a frame with `reject_outer_capture = true`. That is the parser-level guardrail that keeps `Code` non-capturing.

This is a good example of Frothy's style. The rule is enforced early, not hand-waved as "undefined later."

Worked Example

This kind of pseudo-closure is not allowed:

```
to makeAdder with bump [
  fn with x [ x + bump ]
]
```

Why not?

- `bump` is a local of the outer function
- the inner `fn` would need to capture it
- `v0.1` intentionally rejects that model

So if you want reusable changing state, put it in a top-level slot or a top-level owned `Cells` object instead.

What to remember

- The parser resolves names before evaluation.
- Current spoken-ledger surface is sugar over a smaller core.
- `Code` values do not capture outer locals in `v0.1`.
- `repeat`, `when`, and `unless` are parser-level conveniences, not special runtime object kinds.

Common confusions

- "If Frothy is lexical, why not closures yet?" Because lexical name resolution and closures are related but not identical. Frothy has the first without the second.
- "Does `fn` preserve my original source text?" No. It preserves canonical meaning.
- "Are top-level names resolved at parse time too?" The parser chooses slot-read versus local-read structure, but the actual slot value is read at call time.

Invariants

- A local name must resolve to one local index within the current frame.
- Outer-local capture from nested `fn` bodies must be rejected.
- Parser sugar must lower into valid core IR nodes.

- Parse errors must leave the REPL recoverable.

Canonical IR: Node Types, Literals, Links, Rendering, and Why IR Is the Persisted Code Truth

IR stands for *intermediate representation*: the tree form between parsing and evaluation. In Frothy, IR is not a private compiler detail that you can ignore. It is the canonical truth for code.

The node kinds in `src/frothy_ir.h` are compact and revealing:

- `LIT`
- `READ_LOCAL`
- `WRITE_LOCAL`
- `READ_SLOT`
- `WRITE_SLOT`
- `READ_INDEX`
- `WRITE_INDEX`
- `FN`
- `CALL`
- `IF`
- `WHILE`
- `SEQ`

That list tells you what the runtime really understands. If a surface feature is not in that list, it is sugar.

Flow:

```
source text
   |
   v
parser
   |
   v
canonical IR
   |
   +--> surface renderer (`see`)
   |
   +--> core renderer (`core`)
   |
   +--> evaluator
   |
   +--> snapshot codec for persisted code
```

Why persist IR instead of source?

- normalized meaning is easier to restore than original spelling
- snapshots stay smaller and less ambiguous
- `see` and `core` can tell the truth about what will actually run

Code Walk

Read `frothy_ir_render_surface_expr()` in `src/frothy_ir.c`.

That one function shows the whole philosophy. It does not merely pretty-print a stored source string. It pattern-matches the canonical IR and reconstructs a surface form for humans:

- a `WRITE_SLOT` becomes either `name is value` or `set name to value`
- a lowered repeat loop becomes `repeat ... as ... [ ... ]`
- an `if` without an else may render as `when` or `unless`

Then compare that with `frothy_ir_render_code()`, which prints the smaller core tree directly.

Worked Example

Given:

```
demo is fn [ when true [ 42 ] ]
```

`see("demo")` prints:

```
to demo [ when true [ 42 ] ]
```

`core("demo")` prints:

```
(fn arity=0 locals=0 (seq (if (lit true) (seq (lit 42)))))
```

That is not the runtime contradicting itself. It is the runtime showing you the same code at two useful levels:

- human-normalized surface form
- canonical evaluator form

What to remember

- Canonical IR is the code truth Frothy persists.
- Surface syntax can change without changing the deeper IR model.
- `see` is normalized source-like rendering.
- `core` is actual core structure.

Common confusions

- "Why doesn't `see` preserve my exact spacing and wording?" Because it renders meaning, not original text.
- "If `repeat` exists in source, why is there no `REPEAT` IR node?" Because it lowers into locals plus `WHILE`.
- "Does persisting IR make the language less human?" No. It makes the running state more inspectable and less ambiguous.

Invariants

- Every persisted code object must decode into valid canonical IR.
- Renderer output must correspond to the same IR body, not to stale source text.
- Lowered sugar must be semantically faithful to the surface form.

Evaluator Architecture: Frames, Locals, Slot Lookup, Calls, Mutation, Loops, Interrupts, and Safe Points

The evaluator in `src/frothy_eval.c` is a direct tree-walker. That is a good fit for Frothy today because the runtime is small, interruptibility matters, and inspection wants obvious control flow.

Important ideas:

- a **frame** is one call's local storage
- **arity** means how many arguments a function expects

- slot reads are late reads from the stable top-level image
- safe points are places where the runtime checks whether `Ctrl-C` interrupted the program

Code Walk

There are three especially important regions.

First, `frothy_frame_init()` allocates the local frame out of the evaluator's bounded scratch arena.

Second, `frothy_eval_node()` dispatches on IR node kind. This is where you see the real language:

- local read/write
- slot read/write
- index read/write
- function creation
- call
- if
- while
- sequence

Third, `frothy_eval_while()` checks `frothy_poll_interrupt()` on each loop turn. That is the heart of safe interruption. The runtime does not need preemption or threads to stay responsive. It needs good safe points.

One more subtle but important detail: `frothy_eval_program()` converts the internal reset sentinel into a normal top-level `nil` result. That keeps a user action like `wipe()` from tearing through the REPL as a special control-flow explosion.

Worked Example

```
counter is cells(1)
set counter[0] to 0
repeat 2 as i [
  repeat 3 as j [
    set counter[0] to counter[0] + i + j
  ]
]
counter[0]
```

Execution trace in ordinary language:

1. `counter` points at a one-cell `Cells` object.
2. The parser lowers each `repeat` into locals plus a `while`.
3. The evaluator allocates locals for loop limits and indexes.
4. Each body iteration reads `counter[0]`, computes the new value, writes it back, and discards intermediate results.
5. The final result is `9`.

What to remember

- The evaluator is small enough to read in one sitting.
- Slot lookup is dynamic at call time.
- Safe points are explicit.
- Reset paths are deliberately folded back into normal REPL behavior.

Common confusions

- "Why can old code see rebound top-level values?" Because slot reads happen at runtime, not at function-definition time.
- "Why does `repeat` seem like syntax magic?" Because it is parse-time sugar that becomes plain evaluator machinery.
- "Does `while` return the last loop body value?" No. It returns `nil`.

Invariants

- Locals must stay within the frame's declared local count.
- Slot writes must release the old value and own the new value.
- Loop evaluation must poll for interrupts.
- Reset during evaluation must not leave stale temporary ownership behind.

`Cells`: Why They Are Narrow, What They Can Store, How Indexing Works, and Why They Are Top-Level Owned

`Cells` are intentionally narrow. That is not lack of imagination. It is a design decision that keeps the runtime, persistence, and mental model small.

In `v0.1`, `Cells` are the only collection value and they are constrained:

- created with `cells(n)`
- indexed with `frame[i]`

- mutated with `set frame[i] to value`
- intended to be top-level owned, not casually nested everywhere

The stored value classes are deliberately restricted to:

- `Int`
- `Bool`
- `Nil`
- `Text`

Why so narrow?

- snapshot encoding stays simpler
- ownership stays easy to reason about
- you do not accidentally turn `Cells` into a second object graph language

Code Walk

Read three helpers in `src/frothy_eval.c`:

- `frothy_cells_value_allowed()`
- `frothy_read_index_owned()`
- `frothy_write_index_owned()`

`frothy_cells_value_allowed()` is the policy gate. If you try to store `Code`, another `Cells`, or a native binding, it returns `FROTH_ERROR_TYPE_MISMATCH`.

`frothy_read_index_owned()` resolves the `Cells` object, bounds-checks the index, reads the stored tagged value out of `cellspace`, and retains it.

`frothy_write_index_owned()` does the inverse carefully: validate the base, validate the index, validate the stored value class, release the old stored value, then replace it in `cellspace`.

Worked Example

```
frame is cells(2)
set frame[0] to "left"
set frame[1] to true
frame[0]
frame[1]
```

> **RESULT**
> - `"left"`
> - `true`

This works because both stored values are allowed classes. If you try to store a `Code` value instead, the write fails.

What to remember

- `Cells` are narrow by design.
- Indexing is bounds-checked and type-checked.
- The cells payload lives in `cellspace`, not the payload arena.
- Top-level ownership keeps the image model simpler.

Common confusions

- "Why can't `Cells` hold code?" Because Frothy is not trying to be a general object graph language in `v0.1`.
- "Why do `Cells` exist at all if top-level slots are stable?" Because a stable slot often needs a little structured mutable state.
- "Are `Cells` persisted by pointer?" No. Snapshot restore rebuilds them by content.

Invariants

- `cells(n)` requires a positive size.
- Index reads and writes require an integer index in bounds.
- Stored values must be one of the allowed narrow classes.
- Replacing a cell value must release the old stored value.

Persistence: `save`, `restore`, `wipe`, Snapshot Layout, Symbol/Object Tables, Pointer-Free Restore, and Failure Handling

Persistence is one of Frothy's defining traits, but it is deliberately scoped.

Frothy persists the overlay image only. It does **not** persist:

- the current call stack
- in-flight locals
- paused loop position

- arbitrary native process state

That is why the model stays robust.

Snapshot payload layout in plain language:

```
header
  magic = FRTY
  version
  IR version
  payload length
  CRC32

payload
  symbol table
  object table
  binding table
```

The symbol table stores names. The object table stores persistable objects. The binding table says which overlay slots point at which values.

Code Walk

Read `frothy_snapshot_restore()` in `src/frothy_snapshot.c`.

The restore path is deliberately conservative:

1. pick the active snapshot slot
2. read and parse the header
3. verify size bounds
4. read payload bytes
5. verify CRC
6. validate payload structure
7. reset to the base image
8. decode and rebuild the overlay image
9. on any failure, reset back to base again

That “reset back to base again” rule is what makes restore trustworthy. A bad snapshot should not leave you in half-restored limbo.

Then read the top of `src/frothy_snapshot_codec.c`. The codec keeps explicit symbol, object, and decoded-object workspaces. That is how it achieves a pointer-free restore path: persisted data talks in table indexes and literal bytes, not raw process addresses.

Worked Example

```
note is "draft"
frame is cells(1)
set frame[0] to 7
save()
set note to "edited"
set frame[0] to 9
restore()
note
frame[0]
wipe()
note
```

RESULT

- after `restore()`, `note` is `"draft"`
- after `restore()`, `frame[0]` is `7`
- after `wipe()`, `note` is undefined again

What to remember

- Persistence is overlay-only.
- Code is persisted as canonical IR, not as raw source text.
- Native bindings are non-persistable.
- Failed restore returns you to a clean base image.

Common confusions

- "Does `save()` persist everything running right now?" No. It persists the overlay image only.
- "Why does `wipe()` also erase the saved snapshot?" Because in Frothy it means factory-reset the user layer, not merely clear RAM.
- "If code is persisted, why does `see` still look nice?" Because `see` renders from canonical IR back into a human-normalized surface form.

Invariants

- Snapshot payloads must validate before restore starts mutating live state.
- CRC mismatch must abort restore.
- Non-persistable values must make `save()` fail instead of silently degrading.
- Restore failure must leave the system base-only and usable.

Inspection: `words`, `see`, `core`, `slotInfo`, and How Inspection Reflects Canonical IR Rather Than Raw Source

Frothy inspection is not decoration. It is part of the language contract.

The four core inspection tools answer different questions:

- `words()`: what names currently have implementations?
- `see("name")`: what is the normalized surface form of this binding?
- `core("name")`: what is the canonical evaluator form?
- `slotInfo("name")`: is it base or overlay, what class is it, and can it be persisted?

Code Walk

`src/frothy_inspect.c` keeps the implementation pleasingly literal.

`frothy_inspect_resolve_binding()` finds the slot, reads the bound value, figures out its class, and, if the value is `Code`, fetches the IR body, arity, and local count.

From there:

- `frothy_builtin_words()` prints all bound slot names
- `frothy_builtin_see()` prints a header and normalized surface rendering
- `frothy_builtin_core()` prints canonical IR
- `frothy_builtin_slot_info()` prints overlay/base, class, persistability, and ownership

The nice part is that the whole inspection surface is aligned with the runtime model you have already read. There is no second shadow representation.

Worked Example

```
demo is fn [ when true [ 42 ] ]
words()
see("demo")
core("demo")
slotInfo("demo")
slotInfo("save")
```

What each line tells you:

- `words()` confirms the name exists in the live image
- `see("demo")` shows you the human-facing normalized form
- `core("demo")` shows what the evaluator truly runs

- `slotInfo("demo")` tells you it is overlay, code, persistable, user-owned
- `slotInfo("save")` tells you it is base, native, non-persistable, foreign

What to remember

- Inspection reflects the canonical runtime truth.
- `see` is not a source file viewer.
- `core` is the fastest way to check what sugar lowered into.
- `slotInfo` is how you tell base/native from overlay/user state.

Common confusions

- "Why does `see` sometimes print `to ...` even if I used `fn`?" Because it renders the canonical binding view, not your original source spellings.
- "Why does `words()` list board names?" Because filtered board bindings are installed into the base image like any other base slot.

Invariants

- Inspection of a missing name must fail cleanly.
- Code inspection must derive from canonical IR.
- Non-code bindings must still render truthfully as values or metadata.

Interactive Profile: Boot, Safe Boot, REPL, Multiline Input, Prompt Behavior, Ctrl-C, Recovery

Frothy is designed to be lived in interactively. The host shell is not a secondary toy wrapped around a compiler. It is the main user surface.

Boot flow:

```
main
  -> frothy_boot()
     -> platform_init()
     -> runtime init
     -> install base image
     -> print banner
     -> offer safe boot window
     -> maybe restore snapshot
     -> maybe run boot()
     -> enter shell loop
```

Code Walk

Read `src/frothy_boot.c`.

`frothy_boot()` initializes platform and runtime state, installs the base image, prints the startup banner, offers a short `Ctrl-C` safe-boot window, restores a snapshot if one exists, runs `boot()` if it is currently bound to `Code`, then enters `frothy_shell_run()`.

Two design choices are worth noticing.

First, safe boot happens before snapshot restore and `boot()` execution. That means the recovery path is available even when user code or persisted state is broken.

Second, `frothy_boot_run_startup()` records restore and boot errors separately. That keeps start-up failures visible without destroying the ability to reach the prompt.

Then read `src/frothy_shell.c`. The shell has exactly the kind of niceties a small live language should have:

- `frothy>` prompt for complete input
- `..` continuation prompt for incomplete input
- shell command sugar like `show @name`
- `.control` mode entry

Worked Example

A typical manual recovery loop is:

1. start Frothy
2. press `Ctrl-C` during the safe-boot window if you suspect bad saved state
3. run `slotInfo("boot")` or `see("boot")`
4. run `wipe()` if the overlay needs a factory reset
5. reload or resend your project

That is a much better maintainer story than “delete files on disk and hope.”

What to remember

- Safe boot exists to protect the interactive workflow.
- `boot()` is just a slot. It is special by convention and startup behavior, not because it lives in a separate language tier.
- Multiline input is part of normal use.

Common confusions

- "Why is `boot` listed even on a fresh system?" Because the slot is part of the base image, even when it has no user code bound into it.
- "Does `Ctrl-C` always kill the process?" No. In normal operation it is an interrupt signal for the running image.

Invariants

- Startup must always leave the user with a prompt or a clear hard failure.
- Safe boot must skip restore and boot execution.
- Shell command sugar must translate into valid Frothy source.

Control Transport: `.control` Mode, Single-Owner Sessions, Framed Requests/Events, Helper/Editor Story

The plain REPL is great for humans. Tooling needs something more structured. That is what `.control` mode is for.

The control channel is a framed request/event protocol over the underlying link. The key properties are:

- one owner at a time
- request/response sequencing
- output captured as structured events
- explicit idle event marking request completion

The transport underneath uses framed binary messages. The broader Froth tooling stack already uses **COBS** (*Consistent Overhead Byte Stuffing*, a byte-framing scheme that makes delimiter-based serial framing reliable), and Frothy rides on that substrate rather than reinventing it.

Request and event flow:

```
host tool
  |
  | AcquirePrompt() -> newline / Ctrl-C until "frothy> "
  | EnterControl()  -> ".control"
  v
device control mode
  |
  | HELLO / EVAL / WORDS / SEE / SAVE / RESTORE / WIPE / CORE / SLOT_INFO
  v
events
  |
  +-- OUTPUT
  +-- VALUE
  +-- ERROR
  +-- INTERRUPTED
  +-- IDLE
```

Code Walk

Read `src/frothy_control.h` first. The opcode list is the protocol summary:

- `HELLO`
- `EVAL`
- `WORDS`
- `SEE`
- `DETACH`
- `RESET`
- `SAVE`
- `RESTORE`
- `WIPE`
- `CORE`
- `SLOT_INFO`

Then read `frothy_control_start_capture()` and `frothy_control_send_output()` in `src/frothy_control.c`. They temporarily hook platform output so user-visible text becomes structured `OUTPUT` events attached to the active request sequence.

Finally, read `AcquirePrompt()` and `EnterControl()` in `tools/cli/internal/frothycontrol/session.go`. Those methods tell you how the host stays polite:

- first make sure the plain shell is really at a prompt

- then enter control mode
- then exchange framed requests until an `IDLE` event finishes the cycle

Worked Example

`froth send`, `froth build`, and `froth flash` all depend on this machinery.

For example, after an ESP32 flash, the CLI reconnects and sends the staged runtime source through control mode rather than pretending firmware flashing and overlay loading are the same act.

That split is a good maintainer clue: the device image owns the state, and host tools should stay comparatively thin.

What to remember

- `.control` is the structured tool surface.
- Requests complete when the host sees `IDLE`, not merely when output stops.
- Control mode captures output and value separately.

Common confusions

- "Why not just scrape terminal text?" Because tools need explicit values, errors, and completion boundaries.
- "Can two hosts share the same control session?" No. The design is intentionally single-owner.

Invariants

- Session id and sequence id must match on both ends.
- Every handled request must end in `IDLE`, `ERROR`, or `INTERRUPTED`.
- Output capture must be detached cleanly after each request.

Part III: Extending Frothy

FFI Boundary: Native Bindings, Board Slots, Allowed Types, Non-Persistable State, Substrate Reuse

The FFI is where Frothy meets C and board code. It is intentionally narrow.

At the language surface, Frothy only lets native calls exchange a small value set cleanly:

- `Int`
- `Bool`
- `Nil`
- `Text`

It does **not** let the surface FFI freely pass:

- `Cells`
- `Code`
- native values

Why so strict?

- it keeps persistence sane
- it keeps the language model value-oriented
- it prevents board code from quietly widening Frothy into “whatever C can do”

Board bindings enter the base image as named native slots such as:

- `gpio.mode`
- `gpio.write`
- `ms`
- `adc.read`
- `uart.init`
- `uart.write`
- `uart.read`

Pins and constants like `LED_BUILTIN`, `UART_TX`, `UART_RX`, and `A0` arrive as base integer slots.

Code Walk

Read `frothy_ffi_dispatch()` in `src/frothy_ffi.c`.

That function is the translator between Frothy's value model and the inherited Froth substrate FFI. It:

1. validates argument count against the binding metadata
2. pushes Frothy values onto the Froth substrate stack in the expected form
3. calls the native word
4. converts zero or one result back into a Frothy value
5. restores the substrate stack depth even on failure

This is careful bridge code. It is not glamorous, but it is exactly where runtime corruption would begin if you got lazy.

Also notice the filtered name lists at the top of `src/frothy_ffi.c`. Frothy does not automatically expose every board binding that exists in board C code. The language surface is intentionally curated.

Worked Example

```
gpio.mode: LED_BUILTIN, 1
gpio.write: LED_BUILTIN, 1
ms: 50
gpio.write: LED_BUILTIN, 0
adc.read: A0
uart.init: UART_TX, UART_RX, 115200
```

On POSIX:

- GPIO calls print readable traces
- `adc.read` returns a deterministic test value
- `uart.init` returns a small integer handle

On hardware:

- the same names dispatch into the board implementation

What to remember

- FFI is value-oriented at the surface.
- Native values are not persistable.
- Board surface is filtered on purpose.

- Frothy reuses the older substrate here, but it does not inherit the old user-facing stack model.

Common confusions

- “If the board C file implements more names, why do I not see them?” Because Frothy exposes a filtered subset as base slots.
- “Can I save a native binding or native handle?” No. Native values are marked non-persistable.

Invariants

- FFI dispatch must restore substrate stack depth on success and failure.
- Surface FFI conversions must reject unsupported value classes.
- Base board names must be installed as base slots, not overlay slots.

Creating Project FFI in Practice

Project FFI is the extension point for “this project needs one small native thing in addition to the board baseline.”

The manifest surface is:

```
[ffi]
sources = ["src/ffi/bindings.c"]
includes = ["src/ffi"]
defines = { MY_CONSTANT = "42" }
```

And the expected C export is a null-terminated `froth_ffi_entry_t[]` named `froth_project_bindings`.

Hands-on walkthrough

Create the manifest section:

```
[ffi]
sources = ["src/ffi/bindings.c"]
includes = ["src/ffi"]
defines = { MY_CONSTANT = "42" }
```

Then create `src/ffi/bindings.c`:

```
#include "froth_ffi.h"

FROTH_FFI_ARITY(project_magic, "project.magic", "( -- n )", 0, 1,
                "Return a project-specific constant") {
  FROTH_PUSH(MY_CONSTANT);
  return FROTH_OK;
}

const froth_ffi_entry_t froth_project_bindings[] = {
    FROTH_BIND(project_magic),
    {0},
};
```

Then build:

```
froth build
```

What build does today:

- validates that every `[ffi].sources` entry exists, is a `.c` file, and stays under the project root
- validates include directories
- validates define keys
- writes `.froth-build/project_ffi.cmake`
- passes `-DFROTH_PROJECT_FFI_CONFIG=...` into the build

What the generated CMake fragment contains:

- absolute source file paths
- absolute include paths
- compile definitions like `MY_CONSTANT=42`

Code Walk

Read `tools/cli/internal/project/ffi.go` and `tools/cli/cmd/build.go`.

`ResolveFFI()` is intentionally strict. It does not trust project paths by default. It resolves symlinks, rejects files outside the project root, and forces sources to be `.c` files.

Then `prepareProjectFFIConfig()` in `build.go` writes a tiny CMake fragment and the build passes that fragment to the top-level CMakeLists. This is a nice example of thin host tooling: the CLI does path validation and small build staging, then gets out of the way.

What can go wrong

- `[ffi]` without `sources` is rejected.

- source files that are missing, directories, or non-`.c` files are rejected.
- include paths must be directories under the project root.
- define keys must look like valid C identifiers.

One important current-state note for maintainers:

As the code ships today, the build plumbing for project FFI is real, but the runtime auto-install step for `froth_project_bindings` is not wired into the Frothy boot path yet. In practice that means you can compile and link a project-local binding table today, but a name like `project.magic` will not appear in `words()` until that install hook lands.

So this extension surface is best understood as:

- a truthful build seam that already exists
- plus one small runtime integration step that a maintainer can finish cleanly

What to remember

- Project FFI is configured in `froth.toml`.
- The build path is strict and reproducible.
- The current missing piece is runtime installation, not source validation or build wiring.

Creating Board FFI in the Repo

Board FFI is the stronger, already-landed extension path. It is how a board becomes part of the base image.

Each board directory gives you three important artifacts:

- `board.json`
- `ffi.h`
- `ffi.c`

`board.json` defines board metadata and seeded pin constants. For example, the ESP32 DevKit V1 board declares pins such as `LED_BUILTIN`, `A0`, `UART_TX`, and `UART_RX`.

Hands-on walkthrough

A maintainer adding a new board-level capability goes through this flow:

1. add or adjust the native binding in `boards/<board>/ffi.c`
2. export it in the board's binding table

3. decide whether it belongs in the Frothy base-image surface
4. if yes, add its public name to the Frothy filter list in `src/frothy_ffi.c`
5. rebuild and check `words()` plus a real call on POSIX or hardware

The filter step matters. Frothy deliberately ships a narrower board surface than the raw board substrate can expose.

Code Walk

Read three places together:

- `boards/esp32-devkit-v1/ffi.c`
- `boards/posix/ffi.c`
- `src/frothy_ffi.c`

The board files define many substrate-level bindings. Frothy then filters that set through `frothy_board_binding_names[]` and `frothy_board_pin_names[]` before installing names into the base image.

That means “implemented in board C” and “public in Frothy” are intentionally different stages.

This is a good maintainer pattern. It lets you keep extra board helpers or bring-up code around without accidentally widening the public language every time a board author adds a native word.

What can go wrong

- Adding a binding only in `boards/<board>/ffi.c` is not enough if you expect it to become a Frothy base slot.
- Adding too much to the public filter list widens the language surface and the persistence/inspection/testing burden that comes with it.
- Forgetting seeded pins in `board.json` makes board libraries and examples harder to read.

What to remember

- Board FFI is the landed extension path into the base image.
- Public surface is filtered on purpose.
- Seeded board constants are first-class readability tools, not mere metadata.

Tests, Proofs, and Validation Surface: What Is Actually Proved

The test suite is not just regression protection. It is executable teaching material.

High-value test files:

- `tests/frothy_parser_test.c`
- `tests/frothy_eval_test.c`
- `tests/frothy_snapshot_test.c`
- `tests/frothy_shell_test.c`
- `tests/frothy_ffi_test.c`
- `tools/cli/internal/frothycontrol/*.go` tests
- `tools/frothy/proof*.sh`

Code Walk

Read `test_spoken_ledger_surface_and_control_forms()` in `tests/frothy_eval_test.c`.

It is unusually valuable because it exercises the language the way a user thinks about it:

- spoken-ledger bindings and mutation
- `to` and `fn`
- locals
- `cells`
- `repeat`
- `when` and `unless`
- short-circuit boolean logic

Then read `tests/frothy_snapshot_test.c`. That file does more than check happy paths. It forces overflow, bad CRC, format errors, bad names, and failed restore reuse. That is the persistence contract being nailed down in executable form.

Finally, glance at the CLI proof scripts in `tools/frothy/proof*.sh`. Those prove end-to-end stories the unit tests alone cannot.

Worked Example

If you want one maintainer command that gives real confidence, run:

```
make test
```

What that actually buys you:

- parser coverage for surface forms and renderers
- evaluator coverage for value semantics and control flow

- snapshot coverage for save/restore/wipe and failure handling
- shell coverage for command sugar and prompt behavior
- FFI coverage for type conversion and filtering

What it does not magically prove:

- every board-specific hardware behavior on real devices
- every future syntax idea in draft docs
- the absence of all resource-limit edge cases on every target

What to remember

- Tests are part of the language explanation, not just guardrails.
- Snapshot failure-path tests matter as much as success-path tests.
- CLI and control tests prove the host/device boundary, not just C internals.

Common confusions

- “If tests pass, is the board workflow guaranteed on my machine?” Not necessarily. Toolchain and serial environment still matter.
- “Do proof scripts duplicate unit tests?” No. They cover end-to-end paths and release-shape behavior.

Invariants

- New semantics need new tests, not just new prose.
- Every recovery path worth trusting should be exercised somewhere.
- Proof surfaces must stay aligned with the shipped CLI and runtime.

Current Follow-On Queue: What Has Landed, What Is Queued Next, and What Is Still Only Draft Work

The fastest way to stay honest as a maintainer is to separate three buckets in your head.

Landed and real today

- lexical named evaluation model
- stable top-level slots
- base image plus overlay persistence
- canonical IR rendering and persistence

- REPL, safe boot, shell sugar, and `.control` mode
- filtered board FFI surface
- project scaffolding, build, flash, and doctor workflows

Queued next

- the first concrete workspace/image-loading artifact around named slot bundles and IR capsules

That next step matters because Frothy already has the live image and snapshot story. What it still needs is a small, truthful way to move structured image pieces around without pretending whole-file replay is the final answer.

Still draft or follow-on

- broader workspace tooling beyond the first slot-bundle/image-flow cut
- wider language surface beyond the accepted `v0.1` core
- richer runtime data models that would complicate persistence and inspection
- any attempt to widen Frothy casually just because the substrate could do it

The healthiest maintainer posture is this:

- ship the small system completely
- only widen it when the invariants still feel stronger afterward

Frothy is small on purpose. Many omissions are deliberate design choices, not missing polish.